

LLD Requirement Clarification and Class Design

1. Overview

Low-Level Design, or LLD, is the process of converting a problem statement into clear classes, fields, methods, relationships, and APIs.

In interviews, LLD is not only about writing classes. It is about showing that you can think like a software engineer: clarify unclear requirements, identify objects, assign responsibilities, apply SOLID principles, handle edge cases, and design for future extension without over-engineering.

Definition

LLD requirement clarification and class design means:

- Understanding what the system must do.
- Asking questions about scope, constraints, and assumptions.
- Identifying entities and relationships.
- Designing classes with clear responsibilities.
- Defining fields, methods, interfaces, and APIs.
- Handling edge cases and future changes.

Why It Matters

Good LLD helps teams build software that is:

- Easy to understand
- Easy to test
- Easy to change
- Less buggy
- Closer to real user requirements

Bad LLD usually leads to large classes, duplicated logic, unclear ownership, tight coupling, and painful changes.

Where It Is Used in Real Systems

LLD is used while designing:

- Parking lot systems
- Elevator systems
- Snake and ladder games
- Library management systems
- ATM systems
- Food delivery systems
- Ride booking systems
- Splitwise-like expense systems
- Notification systems

- Backend service modules

Why Interviewers Ask About It

Interviewers ask LLD questions to test:

- Whether you can clarify vague requirements.
- Whether you can model real-world entities.
- Whether you understand object-oriented design.
- Whether your classes follow SOLID principles.
- Whether you can handle edge cases.
- Whether your design can support future extensions.
- Whether you can communicate clearly under ambiguity.

2. Core Idea

The core idea of LLD is:

Convert a vague real-world problem into a clean object-oriented design.

Intuition

Imagine someone says:

Design a parking lot.

This is not enough. A good candidate does not immediately write `ParkingLot`, `Vehicle`, and `Ticket`.

Instead, they ask:

- What types of vehicles are supported?
- Are there different spot types?
- Is payment required?
- Is pricing hourly, fixed, or dynamic?
- Are there multiple floors?
- Are reservations supported?
- Are there entry and exit gates?
- Should the system be thread-safe?

Only after clarifying requirements should the design start.

Real-World Analogy

Think of building a house.

Before construction, an architect asks:

- How many rooms?
- How many floors?
- What budget?
- What materials?
- Any parking?

- Any future expansion?

Similarly, before class design, a software engineer asks about requirements, constraints, and extension points.

Small Example

Problem:

Design a library management system.

Clarified requirements:

- Users can search books.
- Members can borrow and return books.
- A book can have multiple copies.
- A member has a maximum borrowing limit.
- Fine is charged for late return.

Possible entities:

- Book
- BookCopy
- Member
- Loan
- Library
- FineCalculator

Possible methods:

- searchBooks(query)
- borrowBook(memberId, bookCopyId)
- returnBook(loanId)
- calculateFine(loan)

Step-by-Step Explanation

1. Clarify requirements.
2. Define scope and assumptions.
3. Identify main entities.
4. Identify relationships between entities.
5. Assign responsibilities.
6. Define classes, fields, and methods.
7. Define service APIs.
8. Apply SOLID principles.
9. Handle edge cases.
10. Discuss tradeoffs and future extensions.

3. Important Subtopics

3.1 Requirement Clarification

Requirement clarification means asking questions before designing.

Why It Matters

LLD problems are intentionally vague. Interviewers want to see whether you can reduce ambiguity before jumping into code.

Example

For "Design an elevator system", ask:

- How many elevators?
- How many floors?
- Can users request up/down direction?
- Is there an emergency mode?
- What scheduling strategy should be used?
- Should the system support maintenance mode?

Common Interview Angle

Interviewers check whether you understand that design depends on requirements.

Poor answer:

I will create Elevator and User classes.

Better answer:

Before designing classes, I want to clarify the number of elevators, request types, scheduling strategy, and failure handling.

3.2 Identifying Entities

Entities are the important objects in the system.

Why It Matters

Correct entities make the design natural and easy to extend.

Example

For a parking lot:

- ParkingLot
- Floor
- ParkingSpot
- Vehicle
- Ticket
- Payment
- Gate

Common Interview Angle

Interviewers expect you to identify both obvious and hidden entities.

Hidden entity example:

- In a library system, **BookCopy** is often more important than **Book**.
- **Book** describes metadata.
- **BookCopy** represents the physical borrowable item.

3.3 Relationships

Relationships define how classes are connected.

Why It Matters

Relationships help decide ownership, lifecycle, and method placement.

Example

In a parking lot:

- A **ParkingLot** has many **Floors**.
- A **Floor** has many **ParkingSpots**.
- A **Ticket** belongs to one **Vehicle**.
- A **Payment** is associated with one **Ticket**.

Common Interview Angle

Interviewers may ask whether a relationship is inheritance, composition, aggregation, or association.

3.4 Responsibilities

Responsibilities define what each class should do.

Why It Matters

A class should have one clear reason to change. This supports the Single Responsibility Principle.

Example

Bad design:

```
ParkingLot
- stores floors
- assigns spots
- calculates price
- processes payment
- prints receipt
```

Better design:

```
ParkingLot
- stores floors
- coordinates parking

SpotAllocationStrategy
- chooses a spot

PricingStrategy
- calculates fee

PaymentService
- processes payment

ReceiptPrinter
- prints receipt
```

Common Interview Angle

Interviewers look for whether your classes become "God classes".

3.5 Classes, Fields, and Methods

Classes represent entities or services. Fields store state. Methods define behavior.

Why It Matters

Good class definitions make the design concrete.

Example

```
class ParkingSpot {
    private String id;
    private SpotType type;
    private boolean occupied;
    private Vehicle currentVehicle;

    public boolean canFit(Vehicle vehicle) { }
    public void park(Vehicle vehicle) { }
    public void vacate() { }
}
```

Common Interview Angle

Interviewers expect enough detail to show implementation thinking, but not unnecessary boilerplate.

3.6 APIs and Services

APIs describe how users or other systems interact with your design.

Why It Matters

Service APIs show the main workflows clearly.

Example

```
interface ParkingService {  
    Ticket parkVehicle(Vehicle vehicle, String gateId);  
    PaymentReceipt exitVehicle(String ticketId, PaymentMode paymentMode);  
    List<ParkingSpot> getAvailableSpots(SpotType type);  
}
```

Common Interview Angle

Interviewers ask:

- What is the input?
- What is the output?
- Who calls this method?
- What happens on failure?

3.7 SOLID Principles

SOLID principles help create maintainable object-oriented designs.

Why It Matters

Interviewers often expect candidates to mention SOLID naturally during LLD.

Example

For pricing:

```
interface PricingStrategy {  
    Money calculateFee(Ticket ticket);  
}  
  
class HourlyPricingStrategy implements PricingStrategy { }  
class FlatRatePricingStrategy implements PricingStrategy { }
```

This follows Open/Closed Principle because new pricing strategies can be added without modifying existing code.

Common Interview Angle

Interviewers may ask:

- Which SOLID principle did you apply here?
- Why use an interface?
- Is this over-engineering?

3.8 Tradeoffs

Tradeoffs mean explaining why one design choice is better for the current requirements, while acknowledging its cost.

Why It Matters

Real design is not about perfect answers. It is about choosing appropriate solutions.

Example

Using a strategy pattern for pricing:

- Advantage: Easy to add new pricing rules.
- Disadvantage: More classes and indirection.
- Good when: Pricing is expected to change.
- Avoid when: Only one fixed price exists and will not change.

Common Interview Angle

Interviewers like candidates who can say:

I am choosing this abstraction because this part is likely to vary.

3.9 Edge Cases

Edge cases are unusual but valid situations that the design must handle.

Why It Matters

Edge cases separate shallow designs from production-ready designs.

Example

Parking lot edge cases:

- No spot available.
- Ticket already paid.
- Vehicle exits without valid ticket.
- Payment fails.
- Same vehicle tries to enter twice.
- Spot becomes unavailable due to maintenance.

Common Interview Angle

Interviewers may ask:

What happens if two vehicles try to take the same spot at the same time?

3.10 Extension Points

Extension points are places where the system can support future changes.

Why It Matters

A good LLD should support likely future requirements without redesigning everything.

Example

In a notification system:

```
NotificationSender
- EmailSender
- SmsSender
- PushSender
- WhatsAppSender
```

Adding WhatsApp should not require rewriting email or SMS logic.

Common Interview Angle

Interviewers ask:

How would you add a new payment method?

Good answer:

I would create a **PaymentProcessor** interface and add a new implementation, keeping existing payment flows unchanged.

4. Real-World Example

Example: Food Delivery Backend

A food delivery app like Swiggy, Zomato, DoorDash, or Uber Eats needs LLD for multiple modules.

Requirements

- Customers can browse restaurants.
- Customers can add items to cart.
- Customers can place orders.
- Restaurants can accept or reject orders.
- Delivery partners can be assigned.
- Payment can be online or cash.

- Order status should be tracked.

Entities

```
Customer
Restaurant
Menu
MenuItem
Cart
CartItem
Order
OrderItem
Payment
DeliveryPartner
DeliveryAssignment
Notification
```

APIs

```
searchRestaurants(location, cuisine)
addItemToCart(customerId, restaurantId, itemId, quantity)
placeOrder(customerId, paymentMode)
acceptOrder(orderId, restaurantId)
assignDeliveryPartner(orderId)
updateOrderStatus(orderId, status)
```

Design Concerns

- A cart should usually contain items from one restaurant only.
- Menu price may change, but order item price should remain fixed after ordering.
- Payment may fail after order creation.
- Delivery partner assignment may need a strategy.
- Notifications should be extensible across SMS, email, and push.

Extension Points

- Add coupon system.
- Add scheduled orders.
- Add subscription plans.
- Add new delivery assignment strategy.
- Add new payment modes.

5. Diagrams / Mental Models

LLD Thinking Flow

```

Problem Statement
  |
  v
Clarify Requirements
  |
  v
Define Scope and Assumptions
  |
  v
Identify Entities
  |
  v
Find Relationships
  |
  v
Assign Responsibilities
  |
  v
Define Classes and APIs
  |
  v
Apply SOLID and Patterns
  |
  v
Handle Edge Cases
  |
  v
Discuss Extensions and Tradeoffs

```

Entity Relationship Example: Parking Lot

```

ParkingLot
  |
  | has many
  v
Floor
  |
  | has many
  v
ParkingSpot
  |
  | occupied by
  v
Vehicle

Vehicle ---> Ticket ---> Payment

```

Responsibility Mapping Table

Requirement	Responsible Class
Store parking floors	ParkingLot
Store spot state	ParkingSpot
Choose a parking spot	SpotAllocationStrategy
Generate ticket	TicketService
Calculate parking fee	PricingStrategy
Process payment	PaymentService
Notify user	NotificationService

Mental Model

Nouns usually become entities.
Verbs usually become methods.
Rules usually become services or strategies.
Things that change often usually become interfaces.

6. Common Interview Questions

1. What is the first thing you should do in an LLD interview?

The first thing is to clarify requirements and scope.

Key Points Interviewer Expects

- Ask functional requirements.
- Ask non-functional constraints if relevant.
- Define assumptions.
- Confirm scope before designing.

Common Mistakes

- Starting class design immediately.
- Assuming all features without asking.
- Over-designing for requirements not mentioned.

2. How do you identify classes in an LLD problem?

Identify important nouns in the problem statement, then validate whether each noun has state or behavior.

Key Points Interviewer Expects

- Look for entities, services, strategies, and value objects.
- Avoid making every noun a class.

- Separate data objects from behavior-heavy objects.

Common Mistakes

- Creating too many unnecessary classes.
- Missing hidden entities like `BookCopy`, `OrderItem`, or `Ticket`.
- Putting all logic in one manager class.

3. How do you decide fields for a class?

Fields should represent the state required by that class to perform its responsibilities.

Key Points Interviewer Expects

- Fields should support required behavior.
- Avoid storing derived data unless needed for performance.
- Keep fields private and expose behavior through methods.

Common Mistakes

- Adding fields without use.
- Making all fields public.
- Storing duplicate state inconsistently.

4. How do you decide methods for a class?

Methods should represent actions the class is responsible for.

Key Points Interviewer Expects

- Methods should align with responsibilities.
- Avoid methods that belong to another class.
- Keep method names business-focused.

Common Mistakes

- Adding generic getters and setters only.
- Putting workflow logic inside entity classes when a service is better.
- Creating large methods that do many things.

5. What is the difference between an entity and a service in LLD?

An entity has identity and state. A service usually contains business logic or workflows that do not naturally belong to one entity.

Type	Meaning	Example
Entity	Object with identity and state	<code>User</code> , <code>Order</code> , <code>Vehicle</code>
Service	Coordinates operations	<code>OrderService</code> , <code>PaymentService</code>

Key Points Interviewer Expects

- Entities model core domain objects.
- Services coordinate workflows.
- Services should not become God classes.

Common Mistakes

- Making everything a service.
- Making entities only data bags.
- Putting all business logic in `SystemManager`.

6. How do you apply the Single Responsibility Principle in LLD?

Give each class one clear reason to change.

Example:

- `Invoice` stores invoice data.
- `InvoiceCalculator` calculates totals.
- `InvoicePrinter` prints invoices.
- `InvoiceRepository` persists invoices.

Key Points Interviewer Expects

- Separate unrelated responsibilities.
- Avoid giant classes.
- Keep change reasons clear.

Common Mistakes

- Saying SRP means "one method per class".
- Splitting classes too much without reason.
- Ignoring business responsibility.

7. How do you apply Open/Closed Principle in LLD?

Design important variation points using interfaces or abstract classes so new behavior can be added without modifying existing stable code.

Example:

```
interface PaymentProcessor {
    PaymentResult pay(Money amount);
}

class CardPaymentProcessor implements PaymentProcessor { }
class UpiPaymentProcessor implements PaymentProcessor { }
```

Key Points Interviewer Expects

- Identify what changes.
- Encapsulate changing behavior.
- Use polymorphism carefully.

Common Mistakes

- Creating interfaces for everything.
- Using inheritance where composition is better.
- Over-engineering simple fixed requirements.

8. What are extension points in LLD?

Extension points are parts of the design intentionally made easy to change or add to.

Examples:

- Payment methods
- Pricing strategies
- Notification channels
- Search filters
- Assignment algorithms

Key Points Interviewer Expects

- Design for likely changes.
- Use interfaces or strategies.
- Mention tradeoff of extra complexity.

Common Mistakes

- Making every part extensible.
- Ignoring expected future changes.
- Not explaining why an extension point is needed.

9. How do you handle edge cases in LLD?

List possible failure and boundary scenarios, then show how the design responds.

Example for parking lot:

- No spot available: return failure or waitlist.
- Invalid ticket: reject exit.
- Payment failure: keep ticket unpaid.
- Spot conflict: use locking or transactional update.

Key Points Interviewer Expects

- Think beyond happy path.

- Mention validation.
- Mention concurrency where relevant.

Common Mistakes

- Only designing happy path.
- Ignoring invalid inputs.
- Not considering state transitions.

10. How much code should you write in an LLD interview?

Usually, write enough class skeletons, method signatures, and important logic to prove your design.

Key Points Interviewer Expects

- Class definitions.
- Important fields.
- Important methods.
- Key interfaces.
- One or two critical flows.

Common Mistakes

- Writing full production code unnecessarily.
- Writing only diagrams with no class details.
- Ignoring method inputs and outputs.

11. What is a God class and why is it bad?

A God class is a class that knows too much or does too much.

Example:

```
FoodDeliverySystem
- manages users
- manages restaurants
- manages cart
- places orders
- processes payment
- assigns delivery
- sends notifications
```

Key Points Interviewer Expects

- It violates SRP.
- It is hard to test.
- It is hard to modify.
- It creates tight coupling.

Common Mistakes

- Creating a **Manager** class for everything.
- Hiding bad design behind one large service.

12. How do you decide between inheritance and composition?

Use inheritance for true "is-a" relationships. Use composition for "has-a" relationships and flexible behavior.

Design Choice	Use When	Example
Inheritance	Stable is-a relationship	Car extends Vehicle
Composition	Object uses another behavior or component	Order has PaymentStrategy

Key Points Interviewer Expects

- Prefer composition for changing behavior.
- Avoid deep inheritance trees.
- Use interfaces for replaceable behavior.

Common Mistakes

- Using inheritance for code reuse only.
- Creating rigid parent-child hierarchies.
- Ignoring strategy pattern when behavior varies.

7. Deep-Dive Questions

1. How would you handle concurrency in an LLD design?

Concurrency matters when multiple users can modify the same resource at the same time.

Example:

Two vehicles trying to book the same parking spot.

Possible solutions:

- Use locks around spot assignment.
- Use database transactions.
- Use optimistic locking with version numbers.
- Keep spot state transitions atomic.

Good answer:

I would make spot allocation atomic. The allocation strategy can suggest a spot, but final reservation must happen through a synchronized or transactional operation.

2. How do you prevent over-engineering while still supporting extensions?

Design extension points only around requirements that are likely to change.

Example:

If payment can be card, UPI, wallet, and cash, use `PaymentProcessor`.

If there is only one fixed payment method and no future variation is expected, a simple method is enough.

Tradeoff:

- More abstractions improve flexibility.
- Fewer abstractions improve simplicity.

3. How do you model state transitions?

Use enums and controlled methods to move between states.

Example:

```
OrderPlaced -> RestaurantAccepted -> Preparing -> OutForDelivery -> Delivered
```

Avoid allowing random state changes.

Better:

```
order.markAccepted();  
order.markPreparing();  
order.markDelivered();
```

Instead of:

```
order.setStatus(anyStatus);
```

This protects invariants.

4. How do you design APIs for an LLD problem?

Start from user workflows.

Example for Splitwise:

- `addUser(userDetails)`
- `createGroup(groupDetails)`
- `addExpense(groupId, paidBy, amount, splitDetails)`
- `getBalance(userId)`
- `settleBalance(fromUser, toUser, amount)`

Good APIs should have:

- Clear input
- Clear output
- Failure handling
- Business meaning
- Minimal exposure of internal details

5. How do you explain tradeoffs in an interview?

Use this format:

```
I chose X because requirement Y may vary.
The benefit is A.
The cost is B.
If the requirement is simpler, I would choose Z.
```

Example:

I used a pricing strategy interface because parking fee rules may change by vehicle type, duration, or day. The benefit is extensibility. The cost is extra classes. If the fee is always fixed, I would keep it as a simple method.

8. Comparison Tables

Requirement Clarification vs Class Design

Aspect	Requirement Clarification	Class Design
Purpose	Understand what to build	Decide how to structure code
Main Activity	Ask questions and define scope	Create classes, fields, methods
Output	Assumptions and requirements	Object model and APIs
Interview Value	Shows product thinking	Shows engineering thinking
Common Mistake	Skipping questions	Creating vague classes

Entity vs Value Object vs Service

Concept	Meaning	Has Identity?	Example
Entity	Domain object with lifecycle	Yes	User, Order, Vehicle
Value Object	Describes a value	No	Money, Address, Location
Service	Coordinates behavior	Usually no	PaymentService, OrderService

Composition vs Inheritance

Aspect	Composition	Inheritance
Meaning	Has-a relationship	Is-a relationship
Flexibility	High	Lower
Best For	Changing behavior	Stable hierarchy
Example	<code>Order</code> has <code>PaymentStrategy</code>	<code>Car</code> extends <code>Vehicle</code>
Interview Tip	Prefer for behavior variation	Use only when natural

Interface vs Abstract Class

Aspect	Interface	Abstract Class
Purpose	Defines contract	Shares contract and partial implementation
Multiple Inheritance	Supported in Java for interfaces	Not supported for classes
State	Usually no instance state	Can have state
Use Case	Payment strategy, notification channel	Base vehicle with common fields
Interview Tip	Use for pluggable behavior	Use when common implementation exists

Manager Class vs Focused Services

Aspect	Manager Class	Focused Services
Responsibility	Often too broad	Narrow and clear
Testability	Hard	Easier
Coupling	High	Lower
Example	<code>SystemManager</code> does everything	<code>PaymentService</code> , <code>OrderService</code> , <code>NotificationService</code>
Interview Tip	Avoid vague managers	Prefer business-specific services

9. Common Mistakes

- Starting class design without clarifying requirements.
- Creating too many classes without purpose.
- Creating one giant manager class.
- Confusing entities with services.
- Making all fields public.
- Adding getters and setters without behavior.
- Ignoring edge cases.
- Ignoring concurrency for shared resources.

- Using inheritance for everything.
- Creating interfaces for everything.
- Not explaining tradeoffs.
- Forgetting state transitions.
- Not defining APIs clearly.
- Missing hidden entities like `BookCopy`, `OrderItem`, or `PaymentTransaction`.
- Designing only the happy path.

10. Edge Cases / Special Cases

Requirement Edge Cases

- Requirement is vague or incomplete.
- Interviewer adds a new feature midway.
- Requirement conflicts with an earlier assumption.
- Non-functional constraints are unclear.

Entity Edge Cases

- One logical concept has multiple physical instances.
 - Example: one `Book` can have many `BookCopy` objects.
- A value should not be modeled as an entity.
 - Example: `Money` usually does not need identity.

API Edge Cases

- Invalid input.
- Duplicate request.
- Failed payment.
- Partial success.
- Retrying the same operation.
- User not authorized.

State Edge Cases

- Order cancelled after payment.
- Elevator under maintenance.
- Parking spot unavailable after being selected.
- Book returned after due date.
- Ticket already paid.

Concurrency Edge Cases

- Two users book the same seat.
- Two vehicles get the same parking spot.
- Two delivery partners accept the same order.
- Payment callback arrives twice.

Extension Edge Cases

- New payment mode.
- New pricing rule.
- New notification channel.
- New vehicle type.
- New user role.
- New search filter.

11. How to Explain in Interview

In an LLD problem, I first clarify the requirements, scope, and constraints. Then I identify the main entities, their relationships, and their responsibilities. After that, I define classes with fields and methods, expose important service APIs, and apply SOLID principles where they naturally fit. I also discuss edge cases like invalid input, concurrency, and state transitions. Finally, I mention extension points and tradeoffs so the design is practical, not over-engineered.

12. Quick Revision Notes

Key Definitions

- LLD: Converts requirements into classes, methods, relationships, and APIs.
- Entity: Object with identity and lifecycle.
- Value object: Object defined by its value, not identity.
- Service: Class that coordinates business workflows.
- Responsibility: What a class owns or does.
- Extension point: A part of design made easy to change.
- Tradeoff: Benefit and cost of a design choice.

Important Points

- Always clarify requirements first.
- Nouns often suggest entities.
- Verbs often suggest methods.
- Changing rules often suggest strategy interfaces.
- Keep classes focused.
- Prefer composition for flexible behavior.
- Mention edge cases and failure handling.

Common Comparisons

Compare	Key Difference
Entity vs Value Object	Identity vs value
Service vs Entity	Workflow coordination vs domain state
Composition vs Inheritance	Has-a vs is-a
Interface vs Abstract Class	Contract vs shared implementation
Clarification vs Design	What to build vs how to build

Must-Remember Facts

- Do not jump directly into classes.
- Do not create a God class.
- Do not make everything abstract.
- Discuss tradeoffs.
- Handle invalid states.
- Show important APIs.
- Design for likely changes, not imaginary changes.

Interview Traps

- "Design X" with vague scope.
- Interviewer changes requirement midway.
- Hidden one-to-many relationship.
- Concurrency on shared resources.
- Duplicate payment or duplicate booking.
- Inheritance vs composition decision.
- Overuse of design patterns.

13. Practice Tasks

Task 1: Design a Parking Lot

Practice:

- Clarify vehicle types.
- Identify spot types.
- Define **Vehicle**, **ParkingSpot**, **Ticket**, **Payment**.
- Add pricing strategy.
- Handle no spot available.

Task 2: Design a Library Management System

Practice:

- Separate **Book** and **BookCopy**.
- Define borrow and return APIs.
- Add fine calculation.
- Handle unavailable books.
- Handle overdue returns.

Task 3: Design an Elevator System

Practice:

- Define elevator states.
- Handle multiple elevators.
- Add scheduling strategy.
- Handle emergency and maintenance mode.

- Discuss concurrency.

Task 4: Design Splitwise

Practice:

- Identify users, groups, expenses, splits, and balances.
- Define split strategies.
- Handle unequal and percentage splits.
- Add settlement.
- Handle rounding issues.

Task 5: Design a Food Delivery Order Flow

Practice:

- Define cart, order, payment, restaurant, and delivery assignment.
- Handle payment failure.
- Handle restaurant rejection.
- Add notification service.
- Discuss extension for coupons.

Task 6: Implement a Mini Notification System

Implement in C++ or Python:

- `NotificationSender` interface.
- `EmailSender`, `SmsSender`, and `PushSender`.
- `NotificationService`.
- Add a new sender without modifying existing senders.

Task 7: Draw an ER-Like Diagram

For any LLD problem, draw:

- Entities
- Relationships
- Ownership
- State transitions
- Important APIs

Task 8: State Transition Practice

Design state transitions for:

- Order lifecycle
- Ticket lifecycle
- Elevator lifecycle
- Payment lifecycle

Example:

```
PaymentCreated -> PaymentProcessing -> PaymentSuccess
                    |
                    v
                PaymentFailed
```

14. Final Cheat Sheet

Core Definition

LLD requirement clarification and class design is the process of converting a vague problem into clear classes, relationships, methods, APIs, edge cases, and extension points.

Why It Matters

It helps build software that is maintainable, testable, extensible, and close to real requirements.

Most Asked Questions

- How do you start an LLD problem?
- How do you identify classes?
- How do you assign responsibilities?
- How do you apply SOLID?
- How do you avoid God classes?
- How do you handle edge cases?
- How do you design APIs?
- How do you support future extensions?
- How do you handle concurrency?
- How do you explain tradeoffs?

Common Comparisons

Comparison	One-Line Answer
Entity vs Value Object	Entity has identity; value object is defined by value.
Service vs Entity	Service coordinates workflows; entity owns state and behavior.
Composition vs Inheritance	Composition is has-a; inheritance is is-a.
Interface vs Abstract Class	Interface defines contract; abstract class can share implementation.
Requirement vs Constraint	Requirement says what to do; constraint limits how it can be done.

One-Line Interview Answer

I clarify requirements first, identify entities and responsibilities, define classes and APIs, apply SOLID where useful, handle edge cases, and explain extension points with tradeoffs.